

WIPRO PYTHON AUTOMATION COURSE

MODULE 1

AUTOMATION WITH SELENIUM

REPORT WORK

Name: - Amit Raj

Course: - Python Automation

Organisation: - Wipro

Technology: - Python, Selenium WebDriver

Enrolment No: - 12023052018002

Stream: - CSBS

Semester: - 7th

DATE: - 11/07/2026

1. Introduction

Under the Wipro Python Automation program, I acquired the basics and advanced knowledge of web automation testing through Python and Selenium WebDriver.

The course enabled me to learn how web application testing is performed manually but could be automated through Python code.

Some of the skills acquired throughout the course include launching browsers, locating web elements, interacting with various controls, managing dynamic web pages, synchronization of automation scripts with the web application, taking screenshots, handling exceptions, reading and writing test data, and doing advanced browser interaction.

Some of the major topics covered in the course include:

- Introduction to Selenium
- Installation and architecture of Selenium WebDriver
- Locators and Object Identification
- Web controls handling
- Browser interactions
- Dynamic Elements and Web Tables
- Waits and synchronization
- Screenshots
- Exception handling
- Data-driven and advanced interactions.

2. Automation With Selenium

2.1 Introduction to Selenium

I came to know that Selenium is an open-source automation framework which enables testers and developers to automate browser activities through programming languages like Python, Java, C# and more.

Selenium is widely used in software testing because it helps automate repetitive browser-based activities. It allows testers to create automation scripts that can interact with web applications in a similar way to a real user. This helps reduce manual effort, improve testing efficiency, and perform repetitive test cases consistently.

The following are some of the repetitive tasks that can be accomplished using Selenium:

- Open a web application
- Enter Username and Password
- Click on buttons
- Make selections
- Verify Text
- Navigate through pages
- Upload files
- Work with multiple browser windows
- Take screenshots

What I learned

I found that Selenium is very much useful for automating web applications especially for functional and regression testing.

The following are some of the components of Selenium:

Selenium IDE

Selenium IDE is an integrated development environment in browser which records and replays user actions as automated test cases. It helps in understanding the basic flow of automation and can be useful for creating and replaying test cases.

Selenium RC

Selenium RC is an old version of Selenium technology through which user interacts with web applications via server-based architecture. Selenium RC represents an earlier approach to browser automation. It used a server-based architecture through which the automation program interacted with the web application. It was an important part of the evolution of Selenium technology.

Selenium WebDriver

Selenium WebDriver is one of the main components of the Selenium technology that is being used in the course. This enables Python programs to communicate directly with the browsers and control browser operations.

Selenium Grid

Selenium Grid allows automated tests to be executed across multiple browsers and multiple machines. This helps in testing web applications in different browser and system environments and is useful for parallel test execution.

3.Selenium WebDriver Introduction and Installation

3.1 Selenium WebDriver

I learned the fundamentals of **Selenium WebDriver** and how Python communicates with a browser through WebDriver.

A typical Selenium automation flow is:

Python Script → Selenium WebDriver → Browser → Web Application

I learned how to create a WebDriver object and use it to control the browser.

For example:

```
from selenium import webdriver
driver = webdriver.Chrome()
driver.get("https://example.com")
driver.quit()
```

Through this exercise, I learned how to:

- Start a browser
- Navigate to a URL
- Perform browser operations
- Close the browser
- End a WebDriver session

3.2 Selenium WebDriver Architecture

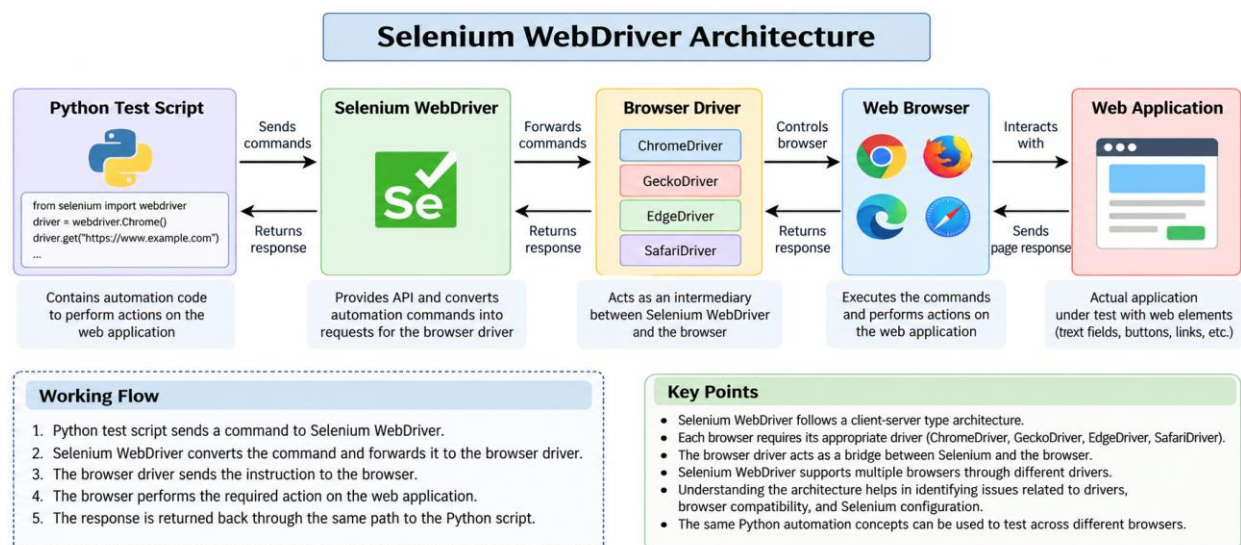
Selenium WebDriver uses a client-server architecture wherein the test script written in Python communicates with the browser using Selenium WebDriver and the corresponding browser driver. Browser driver is a tool that helps the communication between Selenium command and the browser.

This architectural pattern assists in the communication between the automation script and various browsers including Chrome, Firefox, Edge, and Safari. Every browser needs its respective driver or browser automation tool to interpret the commands sent by Selenium WebDriver.

Main Components in Selenium WebDriver Architecture

Some of the components used in Selenium WebDriver architecture include:

- Python Test Script
- Selenium WebDriver
- Browser Driver
- Web Browser
- Web Application



3.3 Installation and Environment Setup

I learned how to configure the Python environment required for Selenium automation.

The major steps included:

1. Installing Python 3
2. Installing Selenium
3. Configuring the browser
4. Creating a Python automation script
5. Launching the browser through Selenium
6. Executing the automation script

Python Installation

The first step was to install Python 3, which is required to write and execute Selenium automation scripts. I learned how to verify the Python installation and ensure that it was properly configured in the system environment.

Selenium Installation

After installing Python, I learned how to install the Selenium package required for browser automation. Selenium provides the necessary libraries and WebDriver functionality to interact with web browsers using Python.

Browser Configuration

I learned how to configure the browser required for automation testing. The appropriate browser and its required driver configuration are important for successfully executing Selenium scripts.

Creating and Executing Automation Scripts

I learned how to create a basic Python automation script using Selenium. The script can be used to launch a browser, open a web application, perform required actions, and execute the automation flow.

I also learned about configuring browser drivers and running automation scripts on different browsers such as:

- Chrome
- Microsoft Edge
- Firefox

This gave me an understanding of **cross-browser testing**.

Cross-browser testing helps verify that a web application works correctly across different browsers. It is important because the same web application may behave differently depending on the browser being used. Through Selenium, I learned how the same automation concepts can be applied to different browsers by using the appropriate browser configuration.

4. Locators and Object Identification

One of the key concepts I gained knowledge about is the concept of locating the web elements.

For any action performed through the Selenium script, it is necessary first to find out the element. Locators are used to uniquely identify the web elements, including text fields, buttons, hyperlinks, check boxes, radio buttons, and drop-downs. There are different ways to locate the elements based on their attributes.

Some of the main locators include ID, Name, Class Name, Tag Name, Link Text, Partial Link Text, CSS Selector, and XPath. Among these, ID and CSS Selector are mostly preferred for locating the elements quickly and accurately, whereas XPath is helpful for more complex web elements. Object identification plays an important role in Selenium scripting.

For example:

```
from selenium.webdriver.common.by import By  
  
element = driver.find_element(By.ID, "username")
```

4.1 Different Locators

I learned how to identify elements using:

- ID
`driver.find_element(By.ID, "username")`
- Name
`driver.find_element(By.NAME, "email")`
- Tag Name
`driver.find_element(By.TAG_NAME, "input")`
- Class Name
`driver.find_element(By.CLASS_NAME, "login-button")`
- Link Text
`driver.find_element(By.LINK_TEXT, "Login")`
- Partial Link Text
`driver.find_element(By.PARTIAL_LINK_TEXT, "Log")`
- CSS Selector
`driver.find_element(By.CSS_SELECTOR, "#username")`
- XPath
`driver.find_element(By.XPATH, "//input[@id='username']")`

4.2 Finding Multiple Elements

I learned how to find multiple web elements on a web page using Selenium WebDriver. Finding multiple elements is useful when a page contains several elements of the same type, such as multiple links, buttons, text boxes, rows, or list items.

Selenium provides the `find_elements()` method to locate multiple elements at the same time. Unlike `find_element()`, which returns a single web element, `find_elements()` returns a list of matching web elements.

For example:

```
from selenium.webdriver.common.by import By
```

```
elements = driver.find_elements(By.TAG_NAME, "input")
```

The returned list can then be used with a loop to perform actions or retrieve information from each element.

```
elements = driver.find_elements(By.TAG_NAME, "input")
```

```
for element in elements:
```

```
    print(element.get_attribute("name"))
```

I also learned that `find_elements()` returns an empty list if no matching elements are found, whereas `find_element()` raises an exception. This concept is useful for handling multiple elements efficiently and performing repetitive operations during automation testing.

5. CSS Selectors

I learned how CSS selectors can be used to identify web elements when simple locators are not sufficient.

For example:

```
driver.find_element(By.CSS_SELECTOR, "#username")
```

I also learned about:

- Attribute selectors
- Child selectors
- Descendant selectors
- Wildcards
- Combining selectors

CSS selectors are useful because they can provide concise and flexible element identification.

6. XPath

XPath was one of the important concepts covered in the training.

I learned that XPath can be used to locate elements based on their position and attributes in the HTML DOM.

6.1 Absolute XPath

Absolute XPath starts from the root of the HTML document.

Example:

```
/html/body/div/form/input
```

I learned that absolute XPath can be fragile because changes in the page structure may cause the XPath to stop working.

6.2 Relative XPath

Relative XPath starts from a specific element and is generally more flexible.

Example:

```
//input[@id='username']
```

I learned to prefer relative XPath whenever possible because it is generally more maintainable.

6.3 XPath Functions

I learned how to use functions such as:

Contains

```
//button[contains(@class,'login')]
```

Starts-With

```
//input[starts-with(@id,'user')]
```

I also learned how to navigate using:

- Parent nodes
- Child nodes
- Sibling nodes
- Ancestor/descendant relationships

This knowledge helped me handle complex web elements.

7. Handling Different Controls on Web Pages

I learned how to interact with different types of web controls using Selenium.

7.1 Handling Buttons

I learned how to locate and click buttons.

```
driver.find_element(By.ID, "submit").click()
```

7.2 Handling Input Boxes

I learned how to enter and clear text from input fields.

```
username = driver.find_element(By.ID, "username")
username.clear()
username.send_keys("testuser")
```

This is commonly used for:

- Login forms
- Search boxes
- Registration forms

- Data-entry forms

7.3 Handling Checkboxes

I learned how to select and deselect checkboxes and verify their state.

```
checkbox = driver.find_element(By.ID, "terms")
if not checkbox.is_selected():
    checkbox.click()
```

7.4 Handling Radio Buttons

I learned how to work with radio buttons and select the required option.

```
driver.find_element(By.ID, "male").click()
```

7.5 Handling Select Boxes

I learned how to work with dropdown/select controls using Selenium's Select class.

```
from selenium.webdriver.support.ui import Select
dropdown = Select(driver.find_element(By.ID, "country"))
dropdown.select_by_visible_text("India")
```

I learned different ways of selecting options, such as:

- Visible text
- Value
- Index

8. Advanced Web Controls

I learned how to automate controls and browser situations that require additional handling.

8.1 Alert Boxes

I learned how to interact with JavaScript alert boxes.

```
alert = driver.switch_to.alert  
alert.accept()
```

I also learned how to:

- Accept alerts
- Dismiss alerts
- Read alert text
- Enter text into prompts where applicable

8.2 Date Pickers

I learned different approaches for handling date picker controls.

The approach depends on how the date picker is implemented in the application.

I learned how to identify the required date elements and select the appropriate date through Selenium.

8.3 File Upload

I learned how to automate file upload fields.

For standard HTML file inputs, Selenium can send the file path directly:

```
upload = driver.find_element(By.ID, "file")  
upload.send_keys("/path/to/file.txt")
```

This is useful for automating document upload functionality.

9. Multiple Windows and Tabs

I learned how to handle situations where clicking a link opens a new browser window or tab.

Selenium provides window handles that can be used to switch between browser contexts.

For example:

```
windows = driver.window_handles  
driver.switch_to.window(windows[1])
```

I learned how to:

- Get available window handles
- Switch between windows
- Perform operations in a specific window
- Return to the original window

10. Keyboard Events

I learned how to simulate keyboard actions using Selenium.

For example:

```
from selenium.webdriver.common.keys import Keys  
element.send_keys(Keys.ENTER)
```

I learned to work with keyboard operations such as:

- Enter
- Tab
- Escape
- Control
- Arrow keys
- Copy/paste-related actions

This is useful when automating applications where keyboard interaction is required.

11. Scrollbar Handling

I learned how to scroll through a web page using Selenium and JavaScript.

For example:

```
driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
```

This is useful when elements are not immediately visible on the screen.

12. Drag and Drop

I learned about automating drag-and-drop operations using Selenium's action capabilities.

The concept is useful for applications containing:

- Upload areas
- Sortable lists
- Interactive dashboards
- Drag-and-drop widgets

13. Handling IFrames

I learned that an iframe contains another HTML document inside the current webpage.

Before interacting with an element inside an iframe, Selenium needs to switch to that frame.

Example:

```
driver.switch_to.frame("frame_name")
```

After completing the operation, I learned to return to the main document:

```
driver.switch_to.default_content()
```

This was important for understanding how Selenium handles different browsing contexts.

14. Finding the State of Web Elements

I learned how to determine whether a web element is:

- Enabled
- Disabled
- Selected
- Visible

For example:

```
element.is_enabled()  
element.is_displayed()  
element.is_selected()
```

This is useful for validation and conditional automation.

15. Getting Text and Attributes

I learned how to retrieve text and attribute values from web elements.

Get text

```
text = element.text
```

Get attribute

```
value = element.get_attribute("value")
```

This is useful for **verification and validation** during automation testing.

For example, after clicking a button, I can retrieve the resulting message and verify whether the expected text is displayed.

16. Handling Dynamic Elements

Modern web applications frequently contain dynamically generated elements.

I learned that element properties such as IDs, classes, and locations may change dynamically.

Therefore, automation scripts should use reliable locators and appropriate synchronization mechanisms.

I learned techniques such as:

- Relative XPath
- XPath contains()
- XPath starts-with()
- CSS selectors
- Explicit waits
- Dynamic element identification

This helped me understand how to make Selenium scripts more robust.

17. Working with WebTables

I learned how to automate HTML tables and retrieve information from rows and columns.

I learned how to:

- Identify a table
- Find table rows
- Find table columns
- Traverse through rows

- Retrieve cell values
- Search for specific data

For example, Selenium can retrieve table rows using:

```
rows = driver.find_elements(By.XPATH, "//table/tbody/tr")
```

This is useful for testing applications such as:

- Employee management systems
- Banking applications
- E-commerce applications
- Reporting dashboards

18. Synchronization and Waits

Synchronization was an important concept because web applications do not always load all elements immediately.

Without proper synchronization, Selenium may try to interact with an element before it is available.

This can result in errors such as `NoSuchElementException` or `ElementNotInteractableException`.

18.1 Implicit Wait

I learned that an implicit wait tells WebDriver to wait for a certain amount of time when searching for elements.

Example:

```
driver.implicitly_wait(10)
```

18.2 Explicit Wait

I learned that explicit waits are used to wait for a specific condition.

Example:

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
element = WebDriverWait(driver, 10).until(
    EC.element_to_be_clickable((By.ID, "submit")))
```

I understood that explicit waits provide more targeted synchronization than using fixed delays.

19. Taking Screenshots

I learned how screenshots can be captured during automation.

Example:

```
driver.save_screenshot("screenshot.png")
```

Screenshots are useful for:

- Debugging failed tests
- Recording test evidence
- Reporting failures
- Analyzing unexpected application behavior

I also learned the concept of creating a **generic screenshot method** that can be reused throughout the automation framework.

20. Exception Handling

I learned how Python exception handling can be used to make automation scripts more stable.

The main concepts covered were:

- try
- except
- finally

Example:

try:

```
driver.find_element(By.ID, "submit").click()
except Exception as e:
    print("Error:", e)
finally:
    driver.quit()
```

I learned that exception handling helps prevent an unexpected error from terminating the entire execution without proper cleanup or information.

21. Advanced Interactions

The advanced interaction section helped me understand how Selenium automation can work with external test data and more complex browser operations.

21.1 Reading Data from Different Sources

I learned how test data can be stored externally rather than hard-coded inside the automation script.

The course covered reading data from:

- Properties files
- CSV files
- XML files
- Excel files
- JSON files

This concept is important for **data-driven testing**.

22. JSON Data Handling

I learned how Python can read and process JSON data.

For example:

```
import json
with open("data.json") as file:
    data = json.load(file)
```

This can be useful for storing:

- Login credentials for test environments
- Test inputs
- Configuration information
- Expected results

23. Excel Data Handling

I learned how Excel files can be used as test-data sources.

The automation framework can read test data from Excel and use that data during test execution.

I also learned the concept of writing test results or other information back into Excel.

This is useful when implementing data-driven automation.

24. Writing Data into Excel and JSON

I learned not only how to read data but also how to write information into external files.

This can be useful for storing:

- Test execution results

- Status
- Error information
- Output data
- Test reports

This improved my understanding of separating **test data from test logic**.

25. Mouse Hover Actions

I learned how to perform mouse-related actions using Selenium.

For example, mouse hover can be performed using the ActionChains API.

```
from selenium.webdriver.common.action_chains import ActionChains
actions = ActionChains(driver)
actions.move_to_element(element).perform()
```

Mouse hover is useful for applications containing:

- Menus
- Submenus
- Tooltips
- Hover-based controls

26. Executing JavaScript Commands

I learned how Selenium can execute JavaScript directly in the browser.

Example:

```
driver.execute_script(
    "arguments[0].click();",
    element )
```

I also learned that JavaScript execution can be useful for:

- Scrolling
- Interacting with certain elements

- Retrieving information
- Performing browser-side operations

However, I understood that JavaScript should be used appropriately rather than as the first solution when normal Selenium interaction is sufficient.

28. Challenges and How I Learned to Handle Them

During Selenium automation, I learned that web applications can behave differently depending on loading time, browser state, element structure, and user interaction.

Some important challenges I learned to handle were:

1. Element not found

I learned to inspect the DOM and choose a suitable locator.

2. Dynamic elements

I learned to use flexible XPath/CSS selectors and synchronization.

3. Timing issues

I learned to use appropriate waits rather than depending on fixed delays.

4. Elements inside iframes

I learned to switch to the appropriate iframe before interacting with its elements.

5. Multiple windows

I learned to use window handles to switch between browser windows or tabs.

6. Unexpected failures

I learned to use exception handling and screenshots to make debugging easier.

29. Overall Learning

The Wipro Python Automation course gave me practical exposure to **web application automation using Python and Selenium WebDriver**.

Initially, I learned the fundamentals of Selenium, WebDriver, browser automation, and element identification. Later, I progressed to more advanced concepts such as dynamic elements, WebTables, synchronization, multiple windows, iframes, screenshots, exception handling, external test data, mouse interactions, and JavaScript execution.

One of the most important things I learned is that successful automation is not only about writing Selenium commands. A good automation script should also be:

- Reliable
- Maintainable
- Reusable
- Readable
- Properly synchronized
- Capable of handling errors
- Independent of hard-coded test data wherever possible

The course therefore helped me build a strong foundation in **Python-based Selenium automation testing**.

30. Conclusion

The Wipro Python Automation course has significantly improved my understanding of automated web application testing.

I learned how to use Python and Selenium WebDriver to automate real-world browser interactions, identify and interact with web elements, handle dynamic web pages, synchronize test execution, manage exceptions, capture screenshots, work with external data sources, and perform advanced browser interactions.

The knowledge gained from this course will help me in developing effective automation scripts and understanding automation frameworks used in software testing projects.

Overall, this training provided me with a strong foundation in **Python + Selenium Web Automation** and helped me understand how automation can improve the efficiency, reliability, and repeatability of software testing.